EXPERIMENT-28

28. Diffie–Hellman Key Exchange Protocol in C

AIM

To write a C program implementing the Diffie–Hellman key exchange protocol and analyze what happens if participants exchange \(x^a\) instead of \(a^x \bmod q\), including possible attacks by Eve.

PROCEDURE

1. Select public values \(a\) and prime number \(q\).
2. Alice chooses secret number \(x\), and Bob chooses secret number \(y\).
3. Alice calculates \(A=a^x\bmod q\), and Bob calculates \(B=a^y\bmod q\).
4. Alice calculates the shared key \(K=B^x\bmod q\), and Bob calculates \(K=A^y\bmod q\).
5. Verify that both participants obtain the same shared key.
6. Analyze the security risks of exchanging \(x^a\) instead of \(a^x\bmod q\).

CODE (C)

```
#include <stdio.h>

long long modpow(long long base, long long exp, long long mod) {
    long long result = 1;
    base %= mod;

    while (exp > 0) {
        if (exp % 2)
            result = (result * base) % mod;

        base = (base * base) % mod;
        exp /= 2;
    }

    return result;
}

int main(void) {
    long long q = 23, a = 5;
```

```
    long long x = 6, y = 15;
    long long A, B, keyAlice, keyBob;

    A = modpow(a, x, q);
    B = modpow(a, y, q);

    keyAlice = modpow(B, x, q);
    keyBob = modpow(A, y, q);

    printf("Public prime q = %lld\n", q);
    printf("Public base a = %lld\n", a);
    printf("Alice's secret x = %lld\n", x);
    printf("Bob's secret y = %lld\n", y);
    printf("Alice sends A = %lld\n", A);
    printf("Bob sends B = %lld\n", B);
    printf("Alice's shared key = %lld\n", keyAlice);
    printf("Bob's shared key = %lld\n", keyBob);

    if (keyAlice == keyBob)
        printf("Key exchange successful.\n");

    return 0;
}
```

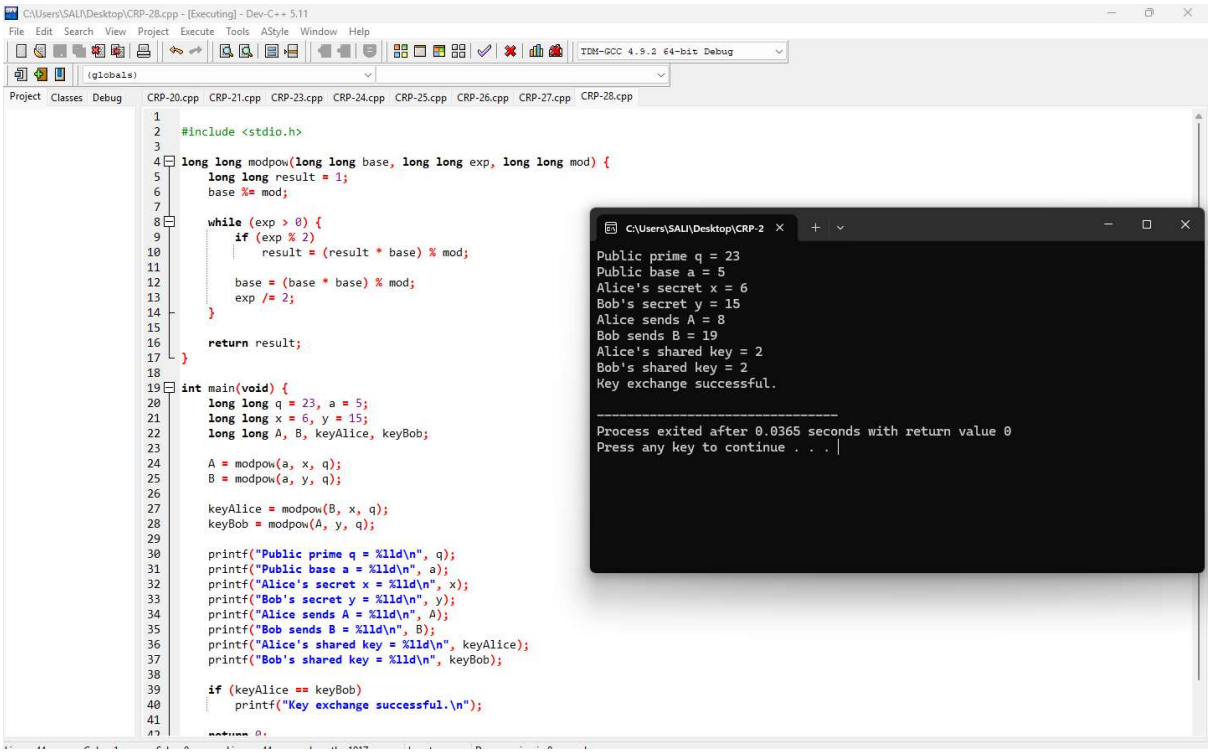
SAMPLE INPUT

The values are assigned in the program.

```
q = 23
a = 5
Alice's secret x = 6
Bob's secret y = 15
```

SAMPLE OUTPUT

```
Public prime q = 23
Public base a = 5
Alice's secret x = 6
Bob's secret y = 15
Alice sends A = 8
Bob sends B = 19
Alice's shared key = 2
Bob's shared key = 2
Key exchange successful.
```



RESULT

The program successfully demonstrates Diffie–Hellman key exchange, with both participants obtaining the same shared key, 2. Standard Diffie–Hellman uses modular exponentiation \(a^x\bmod q\), not simply \(x^a\). In practice, large secure parameters, authentication, and a key-derivation function are needed for security.